Technical Report: Linux File and Directory Permission Audit and Hardening

1. Executive Summary & Strategic Context

This lab demonstrates a Linux file and directory permission audit using POSIX permission bits and symbolic chmod operations. The exercise focuses on identifying and correcting overly permissive access settings, applying the Principle of Least Privilege, and verifying the resulting permissions through command-line output. As part of an enterprise security risk mitigation strategy, the research division initiated a formal access control audit within the `projects` directory tree to enforce a defense-in-depth security posture.

2. Discovery & Initial Security Audit

```
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w---- 1 researcher2 research_team   46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

To establish an initial baseline of asset inventory and permission structures, I performed an exhaustive file system audit of the target directory using the POSIX listing utility. By appending the `-la` flags to `ls`, hidden system dotfiles and comprehensive permission masks were fully exposed alongside standard file metadata. The execution string and corresponding stdout are documented below:

The audit revealed key operational assets: one directory entry (`drafts`), one hidden file (`.project_x.txt`), and five standard project files. Analyzing the 10-character mode string in column 1 established the initial access control posture across file owner, group, and others contexts.

3. Technical Framework: POSIX Permission Bitmask Mechanics

- **Bit 1 (File Type Indicator)**: Identifies the object class. A value of `d` specifies a file system directory container, whereas a hyphen (`-`) indicates a standard binary or text file.
- **Bits 2–4 (File Owner Triad)**: Explicitly defines read (`r`), write (`w`), and execution (`x`) entitlements granted exclusively to the primary file owner. Hyphens indicate unassigned rights.
- **Bits 5–7 (Group Triad)**: Defines read (`r`), write (`w`), and execution (`x`) privileges for secondary identity principals belonging to the mapped group scope.
- **Bits 8–10 (Others Scope Triad)**: Sets global system access boundaries (`r`, `w`, `x`) for security principals in the others scope (outside the file owner and group domain).

4. Permission Audit and Verification: project_k.txt

The initial permissions of project_k.txt were recorded as -rw-rw-r--, granting read and write permissions to the file owner and group, while allowing other users to read the file. These permissions indicate that the file was not world-writable at the time of the initial audit.
To ensure that users outside the owner and group scopes could not modify the file, the following command was used:
`chmod o-w project_k.txt`
This command removes write permission from the others category. However, because the recorded initial permissions already excluded write access for others, executing this command would not change the permission bits in that state.
The resulting permissions should be verified using:
`ls -l project_k.txt`

The shell session below demonstrates the permission adjustment and post-change verification:

```
researcher2@5d738f0f927b:~/projects$ chmod o-w project_k.txt
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-rw--w---- 1 researcher2 research_team   46 Dec  2 15:27 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

If the output remains -rw-rw-r--, it confirms that the file owner and group retain read and write permissions, while others have read-only access. This exercise highlights the importance of comparing initial and final permission states before reporting a security issue as remediated.

5. Remediation Phase 2: Restricting Access Controls on Archived Dotfiles

Hidden files frequently house system parameters, historical logs, or sensitive operational state data. Following the archiving of `.project_x.txt`, policy required adjusting permissions so that both file owner and group scopes hold read-only access while revoking write rights.

The following command sequence documents the multi-attribute permission update applied to the dotfile:

```
researcher2@3213bbc1d047:~/projects$ chmod u-w,g-w,g+r .project_x.txt
researcher2@3213bbc1d047:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec 20 15:36 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec 20 15:36 ..
-r--r----- 1 researcher2 research_team   46 Dec 20 15:36 .project_x.txt
drwx--x--- 2 researcher2 research_team 4096 Dec 20 15:36 drafts
-rw-rw-rw- 1 researcher2 research_team   46 Dec 20 15:36 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec 20 15:36 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec 20 15:36 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec 20 15:36 project_t.txt
researcher2@3213bbc1d047:~/projects$
```

Hidden files are designated by a leading period (`.`) prefix. The initial mode string for `.project_x.txt` was `-rw--w----`, granting read and write access to the owner, write access to the group, and no access to others. To achieve the required read-only posture across owner and group scopes, I executed a compounded symbolic modification: `chmod u-w,g-w,g+r .project_x.txt`. The command-line output validates that the permissions updated to a read-only state for owner and group (`-r--r-----`).

6. Directory Containment & Traversal Privilege Enforcement

In Linux file systems, directory permissions function distinctly from standard file permissions: the read bit (`r`) grants permission to list the names of files inside a directory, while the execute bit (`x`) governs traversal rights (the ability to enter the directory and access file contents or metadata). Compliance guidelines required restricting access to the sensitive `drafts` directory exclusively to the primary owner account (`researcher2`), revoking group traversal privileges.

The terminal output below illustrates the revocation of group traverse rights on the target directory:

```
researcher2@5d738f0f927b:~/projects$ chmod g-x drafts
researcher2@5d738f0f927b:~/projects$ ls -la
total 32
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 .
drwxr-xr-x 3 researcher2 research_team 4096 Dec  2 15:27 ..
-r--r----- 1 researcher2 research_team   46 Dec  2 15:27 .project_x.txt
drwx------ 2 researcher2 research_team 4096 Dec  2 15:27 drafts
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_k.txt
-rw-r----- 1 researcher2 research_team   46 Dec  2 15:27 project_m.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_r.txt
-rw-rw-r-- 1 researcher2 research_team   46 Dec  2 15:27 project_t.txt
researcher2@5d738f0f927b:~/projects$
```

Executing `chmod g-x drafts` successfully revoked traversal entitlements for members of `research_team`. Subsequent verification using `ls -la` confirmed the directory mode mask tightened to `drwx------`, establishing complete containment and preventing unauthorized sub-directory probing.

7. Security Outcomes & Enterprise Impact

This lab demonstrates the use of Linux permission auditing and chmod to manage file and directory access. The verified permission settings for .project_x.txt and drafts illustrate how write and traversal privileges can be restricted according to the Principle of Least Privilege. The project_k.txt experiment also highlights the importance of comparing initial and final permission states before claiming that a security issue has been remediated. Together, these exercises reinforce accurate permission analysis, controlled access, and evidence-based security reporting.